

Liber Usualis — Architecture

1. Overview

Liber Usualis is a data-driven liturgical text generation system for the Roman Catholic Divine Office (Breviary). Given a date and a liturgical hour, it generates the correct text (and optionally chant notation) for praying that hour of the Office, following the rules of a given edition of the Roman Rite (currently 1888).

The system is explicitly designed to be **data-driven**: almost no liturgical logic is hard-coded. Instead, the content is stored in tagged JSON data files, and a tag-lookup system assembles the correct texts at runtime.

1.1. Technology Stack

Layer	Technology
Backend	Python 3.13, Bottle (web framework), Waitress (WSGI server)
Frontend	Alpine.js 3.x, jQuery, Exsurge.js (chant notation)
Templating	Bottle server-side templates (.tpl files)
Data	JSON files with tag sets
Infrastructure	NixOS (at least for testing, prod might move to NixOS eventually)

2. Backend

2.1. Entry Point: `server.py`

The server exposes three primary HTTP endpoints:

- `GET /day?date=YYYY-MM-DD&time=[vesperale|diurnale]` — Returns available liturgical hours and tag sets for the given date.
- `GET /rite?date=...&hour=HOUR&select=...&translation=LANG&privata=...` — Returns fully rendered liturgical content for a specific hour.
- `GET /calendar` — Returns the calendar skeleton for navigation.

The server also proxies chant notation files from external sources under `/chant/...`.

2.2. Calendar & Prioritization: `prioritizer.py` and `calendar/`

Before any text is fetched, the backend must determine **which** feast or season governs a given date. This is handled in two stages.

2.2.1. Stage 1: Raw Calendar Lookup

`calendar/` contains a full liturgical calendar system, including lunar and solar calculations. Given a date, `datamanager.getdate(day)` returns the base liturgical composition: which saint's feast it is, which week of the season, the day of the week, etc. These are expressed as sets of tags.

2.2.2. Stage 2: Prioritization

The raw calendar data is then passed through the **prioritizer**, which applies rule tables to resolve conflicts between feasts and handle the many special cases of the Roman Rite (e.g., a feast displacing a ferial office, or two feasts coinciding).

Rule tables live in `data/{root}/`:

- `tabella-diurnalis.json` — Rules for the daytime office
- `tabella-vesperalis.json` — Rules for Vespers
- `tabella-martyrologii.json` — Rules for the Martyrology reading

Each rule specifies:

- **Restrictions:** `include` (tags that must be present) and `exclude` (tags that must be absent) to match a situation.
- **Operations:** what to do when matched — `crea` (create tag set), `combina` (merge), `mutate` (modify in place), `duplicate` (create modified copy), `dele` (delete), or `errora` (raise conflict).

The output of the prioritizer is a list of `frozenset` objects, one per commemorated service (primary feast, commemorations, omissions, votive options).

2.3. Data Loading: `datamanage.py`

Once the relevant tags are known, `getpile(root, tagset)` scans `data/{root}/tagged/` for JSON files that contain any of the relevant tags, loads them, and returns a flat list of all candidate entries. Results are LRU- cached (maxsize=1024).

2.4. Liturgical Generation: `breviarium.py`

The core of the backend. The `process(root, item, selected, alternates, pile)` function recursively assembles liturgical content.

2.4.1. Core algorithm

1. If `item` is a bare set of tags, treat it as a search request: find the best-matching entry in `pile` whose tags are a subset of the query (see [2.4.2](#) below).
2. If `item['datum']` is a list, recursively process each element.
3. If `item['datum']` is a string, return it as-is (possibly substituting saint names, abbreviated as `N.`).
4. Handle special directives: `choose` (alternatives), `with` (supplements), `from-tags` (secondary lookup source).

The result is a nested JSON structure mirroring the hierarchy of the liturgical hour (hour → sections → individual items).

2.4.2. Search & Discrimination

`breviarium.search(query, pile)` finds the best match for a tag query:

1. Collect **candidates**: all entries in `pile` where the entry's tags are a subset of the query.
2. Score each candidate using `discriminate()` and the `discrimina/general.json` ranking table (see [4.6](#)).
3. Return the highest-scoring candidate. If there is a tie, prefer the entry with more tags (more specific). If still ambiguous, raise an error.

Post-processing via `managesearch()` handles antiphon editing modes (`intonata` , `repetita` , `pars`).

2.5. Psalm Resolution: `psalms.py`

Psalm numbers are resolved separately. Queries containing `/psalmi-...` tags are intercepted and fetch the psalm text directly from `psalterium/` files, bypassing the normal search.

2.6. Localization: `localization.py`

Locale is detected from the `Accept-Language` HTTP header. Supported locales: `la` (Latin), `en` (English), `de` (German). When a translation is requested, the server adds a language tag to search queries and loads translation files from `data/{root}/translations/{locale}/`.

3. Frontend

3.1. Entry Point: `web/templates/pray.tpl`

The main page is a Bottle server-side template. It sets up the Alpine.js reactive component that drives all interactivity.

3.2. State Management: Alpine.js (`pray.tpl`)

The Alpine.js `x-data` block holds the application state:

Property	Description
<code>liturgylist</code>	Available hours for the day (from <code>/day</code> response)
<code>date</code>	Current date (timezone-aware)
<code>time</code>	<code>vesperale</code> (evening) or <code>diurnale</code> (daytime)
<code>hour</code>	Currently selected liturgical hour
<code>choral</code>	Whether choral formatting is active
<code>chant</code>	Whether chant notation is shown
<code>recitation</code>	Display mode: choral / recto-tono / private
<code>translation</code>	Language preference
<code>ambit</code>	Current ritual template (see below)

Preferences (`choral` , `chant` , `recitation` , `translation`) are persisted to `localStorage` .

3.3. Ritual Templates: `web/resources/js/ritegen.js`

An **ambit** is a template that defines which elements appear in a given type of office. Several ambits are defined:

- `fullambit` — Standard daily office (Matutinum through Completorium)
- `opbmvambit` — Little Office of the Blessed Virgin Mary
- `defunctambit` — Office of the Dead
- `gradualambit` — Gradual Psalms
- `penitentialsambit` — Penitential Psalms

Each ambit contains `RiteItem` objects specifying:

- `what` — The liturgical item type (antiphon, psalm, hymn, etc.)
- `where` — Where to source it (`diei` = primary, `officium-parvum-bmv` = votive, etc.)
- `always` — Whether the item always displays regardless of tags

The `ritelist()` function filters ambit items against the tags present in `daytags` (returned by `/day`), hiding items that have no matching content. `defineambit()` selects the appropriate ambit based on the requested office.

3.4. Event Handlers: `web/resources/js/pray.js`

Function	Description
<code>updateDay()</code>	Fetches <code>/day</code> ; populates hour list and <code>daytags</code>
<code>updateRite()</code>	Fetches <code>/rite</code> ; calls <code>render()</code> to display content
<code>slideHour(id)</code>	Changes the selected hour
<code>setTime(time)</code>	Switches between vesperale and diurnale
<code>incrementHour()</code>	Advances to the next hour, crossing day boundary if needed
<code>setRecitation()</code>	Changes display mode
<code>dopanelsize()</code>	Manages responsive side panels for featured artwork

3.5. Chant Rendering

When chant notation is enabled, the frontend requests chant files (GABC format) via `/chant/...` (proxied by the server). These are rendered in-browser by `exsurge.js`.

3.6. HTTP API (Frontend → Backend)

Endpoint	Parameters	Returns
<code>/day</code>	<code>date</code> , <code>time</code>	JSON: available hours, tag sets, feast names
<code>/rite</code>	<code>date</code> , <code>time</code> , <code>hour</code> , <code>select</code> , <code>translation</code> , <code>privata</code>	JSON: full nested liturgical content
<code>/kalendar</code>	—	Calendar skeleton for navigation
<code>/chant/...</code>	<code>path</code>	Proxied chant notation file

4. Tagging System

Tags are the central abstraction. Every piece of content is identified by a set of tags, and every request is resolved by searching for the best tag match.

4.1. What a Tag Is

A tag is a lowercase, hyphen-separated string. Examples:

- `immaculata-conceptio-bmv` — The feast of the Immaculate Conception
- `matutinum` — The liturgical hour of Matins
- `antiphona-i` — The first antiphon position within an hour
- `in-tempore-adventus` — The season of Advent
- `duplex-majus` — The feast rank "Greater Double"
- `primarium` — Primary (as opposed to commemorated) service
- `privata` — Private recitation mode

4.2. Tag Categories

Tags are organized into categories defined in `data/{root}/categoriae/`:

- `positionales.json` — Mutually exclusive positional tags (hours, antiphon positions, psalm selections). Only one tag from each group may apply at once.
- `temporale.json` — Liturgical season and feast-rank tags.
- `proprium-sanctorum.json` — Individual saint feast tags.
- `objecta.json` — Content-type tags (antiphon, psalm, hymn, reading, etc.).

Tags starting with `/` inside category files are **references** to other categories, and are expanded to all tags in that category at runtime.

4.3. Tag Storage in Data Files

Content is stored in JSON files under `data/{root}/tagged/`, organized hierarchically:

```
data/breviarium-1888/tagged/
├── proprium-sanctorum/      (specific saint feasts, by month)
│   └── december/immaculata-conceptio-bmv.json
├── proprium-de-tempore/    (temporal cycle: seasons, Sundays)
│   ├── in-tempore-adventus/
│   ├── in-tempore-quadragesimae/
│   └── hebdomadae-*.json
├── commune-sanctorum/     (common texts shared by multiple saints)
├── votiva/                 (votive offices)
├── psalterium/            (psalm texts)
└── formulae.json          (fixed formulas: doxologies, responses, etc.)
```

Each entry in a JSON file is an object with:

- `tags` : A list of tags (or a list of tag lists for entries matching multiple distinct contexts).
- `datum` : The content (text, or a list of sub-items for structured content).
- `src` (optional): Reference to a chant source (`gregobase/12345` , `nocturnale/A1F2R1` , etc.).

Example entries:

```
{
  "tags": ["immaculata-conceptio-bmv", "nomen"],
  "datum": "Immaculatæ Conceptionis B.M.V."
}

{
  "tags": ["immaculata-conceptio-bmv", "matutinum", "antiphona", "antiphona-i"],
  "src": "gregobase/2922",
  "datum": "Tota pulchra * es María, et mácula originális non est in te."
}
```

An entry with multiple tag sets (appears in multiple contexts):

```
{
  "tags": [
    ["hebdomada-i-adventus", "feria", "feria-ii", "matutinum", "responsorium", "responsorium-i"],
    ["hebdomada-ii-adventus", "feria", "feria-ii", "matutinum", "responsorium", "responsorium-i"]
  ],
  "src": "nocturnale/A1F2R1",
  "datum": "..."
}
```

4.4. Tag Generation at Runtime

When a user requests a date, the pipeline generates tags in three steps:

1. **Calendar lookup** — `kalendar` returns base tags for the date (saint's name, week of season, day of week, etc.).
2. **Prioritization** — `prioritizer` applies rule tables to resolve conflicts, producing a list of `frozenset` objects (one per service: primary, commemoration, omission).
3. **Implication (sequentes)** — `kalendar/data/sequentes.json` defines derived tags: if certain tags are present, additional tags are automatically added. For example, `in-tempore-passionis` implies `in-tempore-quadragesimae`.

4.5. Tag Matching (Search)

To find content for a specific liturgical item, the system builds a **query**: a set of all currently relevant tags (from the calendar) plus the specific item-type tags (e.g., `matutinum`, `antiphona`, `antiphona-i`).

A data entry **matches** the query if the entry's tags are a **subset** of the query. This means: an entry tagged `["immaculata-conceptio-bmv", "antiphona-i"]` will match any query containing both of those tags, regardless of other tags in the query.

4.6. Discrimination Tables

Because multiple entries may match a query (some specific, some general), the system ranks them using **discrimination tables** (`data/{root}/discrimina/`).

`discrimina/general.json` defines a priority ordering of tag combinations. The `discriminate()` function scores each candidate entry by checking, in priority order, which tags it has. The result is a bitmask: higher-priority matches set higher bits, so the numeric value directly represents overall specificity.

The entry with the highest score wins. If scores are tied, the entry with more tags (more specific) is preferred.

4.7. Contradiction Resolution

When positional tag groups (from `categoriae/positionales.json`) produce contradictions — e.g., a search simultaneously implies both `antiphona-i` and `antiphona-ii` — the system removes the conflicting tags from the query before searching. This prevents ambiguous matches.

4.8. User-Controlled Tag Modification

The `/rite` endpoint accepts parameters that adjust the active tag set:

- `select=officium-parvum-bmv` — Replace the primary office with the Little Office of the BVM.
- `privata=privata` — Add the `privata` tag, changing formatting rules (e.g., simplified antiphon handling).
- `translation=en` — Add a language tag, causing the system to load translation overlay files.

5. Data Flow: End-to-End Example

Request: `GET /rite?date=1888-12-08&hour=matutinum`

1. `server.py` receives request
 - └ parse date (Dec 8 = Immaculate Conception), hour = matutinum
2. `prioritizer.getdiurnal("1888-12-08")`
 - └ returns `[frozenset({'immaculata-conceptio-bmv', 'duplex-majus', 'primarium'})]`
3. `breviarium.process()` begins
 - └ for each item in the matutinum ambit template:
 - a. Build query: primary tags $\cup$ positional tags
e.g. `{'immaculata-conceptio-bmv', 'duplex-majus', 'matutinum', 'antiphona', 'antiphona-invitorium'}`
 - b. Load pile: relevant JSON files from tagged/
(`proprium-sanctorum/december/immaculata-conceptio-bmv.json`, `formulae.json`, `psalterium/`, ...)
 - c. `search(query, pile)`: find entry whose tags $\subseteq$ query with highest `discrimina` score
 - d. Recurse into sub-items if datum is a list
4. Assemble and return JSON:

```
{
  "rite": { "tags": ["matutinum"], "datum": [
    {"tags": ["antiphona-invitorium"], "datum": "Immaculátam Conceptionem..."},
    {"tags": ["psalmi"], "datum": [...]}],
    ...
  ]},
  "usedprimary": ["immaculata-conceptio-bmv", "duplex-majus"],
  "usednames": ["Immaculatæ Conceptionis B.M.V."]
}
```

5. Frontend render() converts JSON to HTML
└─ optionally fetches and renders chant notation via exsurge.js

6. Key File Index

File / Directory	Purpose
server.py	HTTP routing and request handling
breviarium.py	Core liturgical text generation engine
datamanager.py	Data loading, caching, JSON/frozenset conversion
prioritizer.py	Calendar conflict resolution and tag generation
psalms.py	Psalm number resolution
localization.py	Locale detection and i18n
kalendar/	Full liturgical calendar with solar/lunar calculations
web/templates/pray.tpl	Main page template (Alpine.js root component)
web/resources/js/ritegen.js	Ritual hour templates (ambits) and rendering logic
web/resources/js/pray.js	Event handlers and state management
web/locales/{locale}/	UI string translations
data/breviarium-1888/tagged/	All tagged liturgical content (JSON)
data/breviarium-1888/categoriae/	Tag classification and mutual-exclusion groups
data/breviarium-1888/discrimina/	Search ranking tables
data/breviarium-1888/translations/	Content translations
kalendar/data/sequentes.json	Implied-tag rules
data/breviarium-1888/tabella-diurnalis.json	Daytime office prioritization rules
data/breviarium-1888/tabella-vesperalis.json	Vespers prioritization rules